GUARDRAIL CONSOLE — PROJECT DEMO REVIEW

The flow, the techniques, and where the agents sit

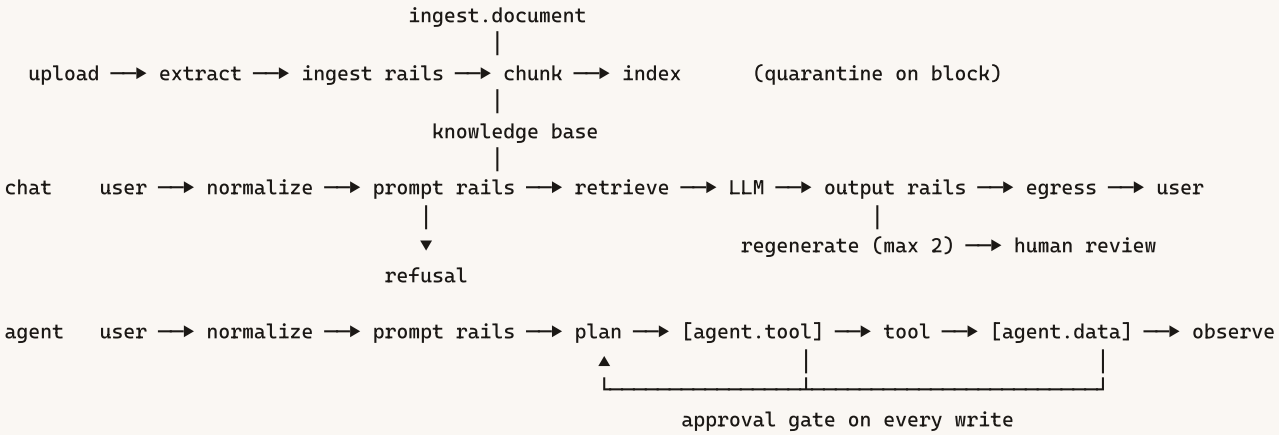
An enterprise guardrail stack for LLM applications, built on Claude. This review walks the whole pipeline end to end — ingestion, chat, and the agent path — names the concrete technique behind every checkpoint, and separates the two places "an agent" means something different in this codebase: the tool-calling `AgentRunner`, and the deeper reasoning layer of `GuardrailSupervisor`, `Supervisor`, and its six specialists.

```
Deterministic code first, a local model second, a Claude judge only when both are undecided
Every number below is measured on a running deployment, not estimated
Prepared for a project demo / review walkthrough
```

What is in here

1 · The system in one page

The Guardrail Console sits in front of, and behind, an LLM. Every request is evaluated by **rails** — small, independently testable checks — before the model is called and again on what it produces. Nothing about a rail changes between checkpoints; what changes is the **surface** it runs on, and how strict the surface makes it.



Masked values — an SSN, a card number, a name — travel through the model as opaque **vault tokens** and are decrypted only at egress, for a caller entitled to see them. The model itself never sees a raw identifier.

Five guardrail families

FAMILY	WHAT IT DOES	ENGINE
Content	hate, violence, insults, misconduct, self-harm, sexual, prompt injection	pattern layer → local classifier → Claude judge
Word	profanity, custom terms/phrases, allowlist exemptions	Aho–Corasick, one pass over the whole lexicon
Sensitive info	email, phone, SSN, card, IBAN, Aadhaar, PAN, IP, DOB, custom regex	regex + checksum gate, AES-256-GCM vault
Grounding	factual consistency and relevance vs. retrieved context	Claude judge, claim-level
Policy	severity matrix, verdict precedence, latency budget, fail mode	YAML + registry

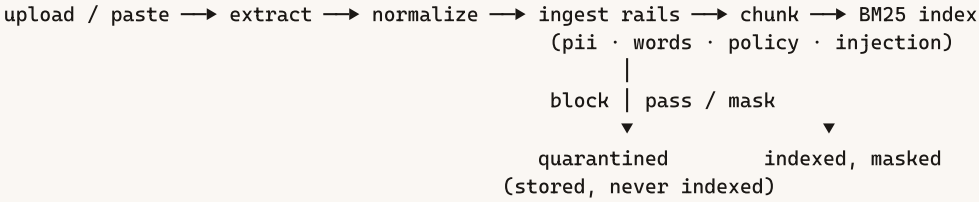
Plus two subsystems with their own trust boundary

	WHAT IT DOES	ENGINE
Ingestion	extract, scan, mask, chunk, index — or quarantine	BM25 index · rails at `ingest.document`
Agent & tools	tool calls with a rail on the arguments and on the result, an approval gate on every write	Claude tool use · `agent.tool` / `agent.data`

The one rule everything else follows. Every checkpoint is the same function, `Engine.evaluate(text, surface)` . A rail never knows which surface it is on — the surface decides only *which* rails run and *how strict*. Adding a new trust boundary is a new surface, never a new pipeline.

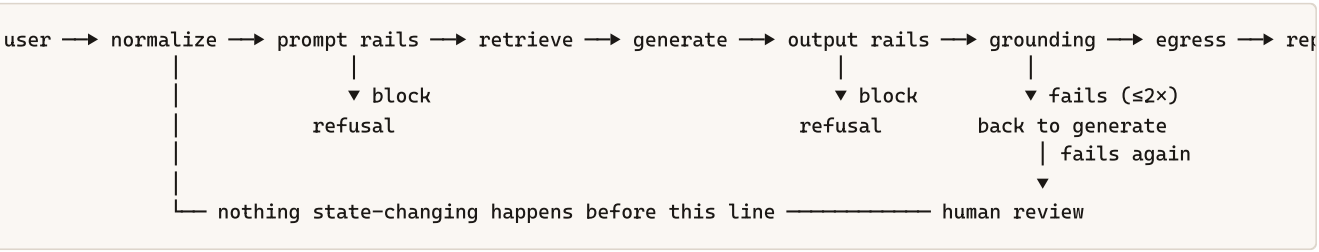
2 · The end-to-end flow

2.1 · Ingestion — a document becomes searchable, or is quarantined



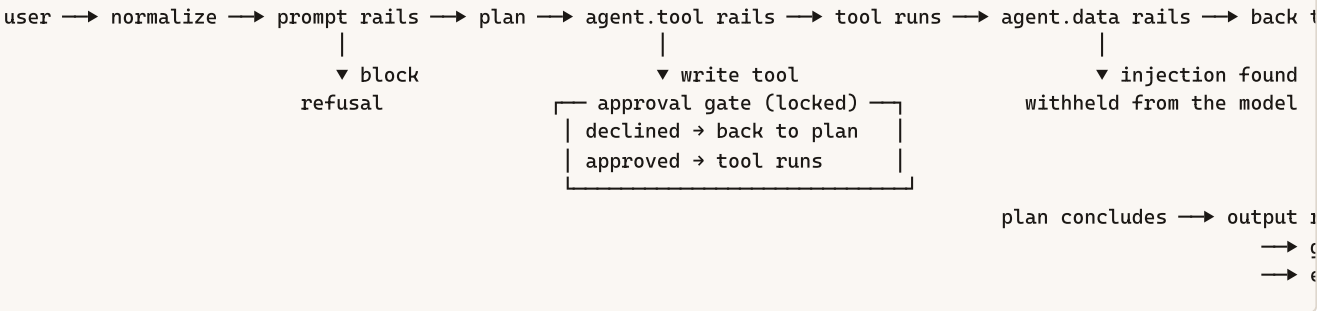
Injection scanning is **locked on** at this surface regardless of the severity matrix — indirect injection through an uploaded document is the whole reason ingestion is its own trust boundary. Masking always happens **before** indexing, and a blocked document is quarantined, not flagged: indexing it with a warning would make retrieval safety depend on someone remembering to check the warning, forever.

2.2 · A chat turn



A failed output rail returns the model to another attempt, **never** straight to the user — only a *second* failure surfaces a person, through the human-review queue.

2.3 · An agent turn



Two surfaces exist here and nowhere else in the chat path, and neither is optional: `agent.tool` inspects the arguments the model produced *before* the call runs; `agent.data` inspects what came back *before* the model is allowed to read it. `agent.tool_result_trust` is locked — a tool result is attacker-reachable text the same as a user message is.

3 · The chat turn, stage by stage

Eleven numbered stages, every one timed by the tracer itself rather than by the rail — so a rail cannot forget to report a number, or report one it made up.

#	STAGE	WHAT HAPPENS
1	Ingress	session bind, vault opened (AES-256-GCM), tracer starts, `request_id` minted
2	Normalize	NFKC → strip invisibles → homoglyph fold → collapse whitespace. Locked — see §4.
3	Prompt rails	every enabled rail for `user.prompt` runs concurrently under one shared deadline
4	Policy decision	`precedence()` — block ▶ mask ▶ flag ▶ pass. Most restrictive wins, always.
4b	Close-call review	a Claude adjudicator revisits only the scores sitting within a narrow margin of their own threshold
5	Retrieval	BM25 over the indexed chunks, top `k`, gated by term coverage
6	Retrieval rails	what came back is untrusted too — scanned, masked, or dropped before the model sees it
7	Generation	system prompt + numbered context + the masked question, non-streaming
8	Output rails	the same families run again, at `llm.response`'s own severity
9	Grounding	a Claude judge scores consistency (≥0.50) and relevance (≥0.35) against the retrieved chunks, separately
10	Egress	vault tokens decrypted for an authorised caller; the audit entry is hash-chained onto the log
11	Review trigger	`policy.human_review.trigger` consulted once, on every exit path — including blocked ones

> **Two failure modes, handled once, in one place.** A rail that *raises* follows `policy.fail_mode` (fail-closed by default). A rail that does not *finish* inside the latency budget is recorded as `BLOCK` regardless of `fail_mode` — `policy.timeout_behavior` is a locked safety invariant, because "we did not finish checking" is not the same claim as "there is nothing here."

4 · The techniques and methods

The design principle behind every rail: decide as cheaply as possible, and only pay for a model call when a cheaper layer is genuinely undecided.

```
structural gate     0.2ms      no capitalised word? no model is called at all
deterministic       0.1ms      injection patterns, checksums, the word automaton
local model       90–300ms     a confident hit may block — never clear
Claude judge       1.8–4s      everything still undecided
```

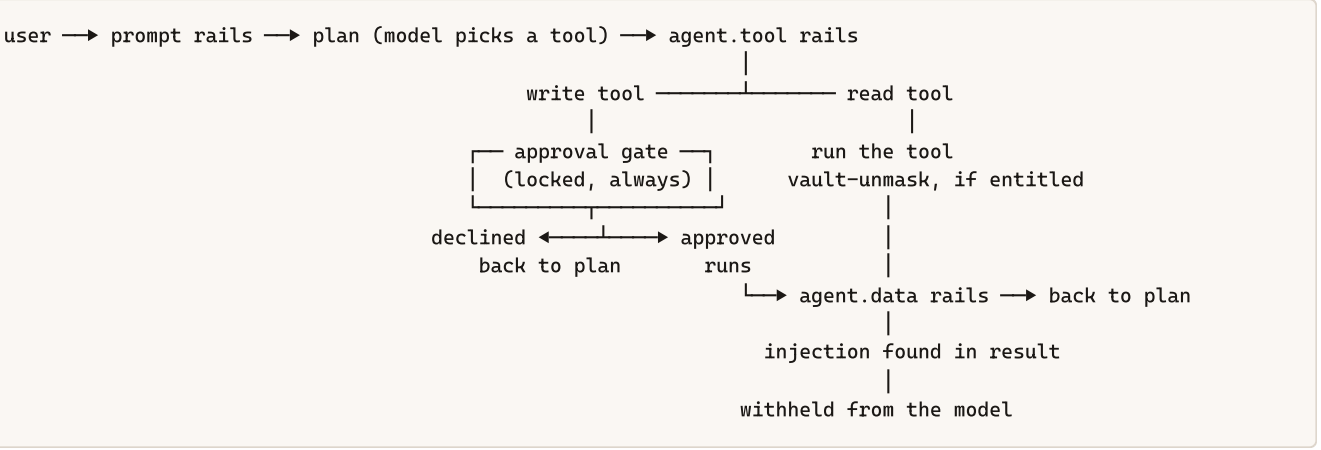
The rule the middle tier follows: a local model may end a request early only by *blocking* it, never by clearing it. "This model found nothing it was trained to find" and "there is nothing here" are different claims, and only one is safe to act on.

CHECKPOINT	CONCRETE TECHNIQUE
Normalization	NFKC → invisible-character strip → homoglyph fold → whitespace collapse. Locked on; homoglyph substitution is the single most common lexical-filter bypass.
Word / lexicon	**Aho–Corasick** automaton — one pass over the input regardless of lexicon size, O(n + matches).
PII detection	regex recognizers, gated by a **checksum** (Luhn for cards, Verhoeff for Aadhaar, IBAN mod-97, SSA range rules) before anything is masked. The checksum gate is what keeps false positives low enough that nobody disables the rail.
PII masking	a random, per-occurrence **vault token**, encrypted with **AES-256-GCM**; decrypted only at egress for an entitled caller. A stable token would be a stable identifier an observer could correlate across users.
Names & addresses	a structural gate (no capitalised candidate → skip), then local **Presidio NER** (~27ms), then a Claude judge — asked only when NER comes back empty. Every span is re-checked against the source text before masking.
Prompt injection	a deterministic pattern layer first; a confident hit (≥0.85) short-circuits the Claude judge entirely — most of what keeps median latency down.
Content safety	a toxicity classifier under a Claude judge that scores six categories in one structured call, with no conversation history (history is attacker-controlled).
Retrieval	**BM25** ranking over indexed chunks, gated separately by **term coverage** — BM25 answers "which chunks rank highest," coverage answers "is any of this actually about the question."
Grounding	a Claude judge scoring the answer against only the chunks actually retrieved, on two independent numbers (consistency, relevance) rather than one blended score.
Verdict combination	fixed **precedence** — block > mask > flag > pass. Never a vote or an average: one restrictive rail always outranks any number of permissive ones.
Close-call review	a Claude adjudicator may raise a verdict freely, but may only lower one that triggered it, only above a confidence floor, and never below `flag` — and never for a deterministic rail or a timed-out one.
Concurrency	every rail in a stage runs in a `ThreadPoolExecutor` against one shared deadline; a stage costs its slowest rail, not the sum of all of them.
Tracing & audit	a tracer external to every rail records timing and verdict; the audit log is **hash-chained**, one JSON line per request, so tampering with any entry breaks verification from that point forward.

Masking composes deliberately. Each masking rail rewrites the text it was actually given, and a later rail re-runs against what the previous one already produced — so a blocked word sitting next to an SSN gets both replacements, not just whichever rail happened to run last.

5 · Where the agents work — layer one: `AgentRunner`

This is the tool-calling agent behind the chat assistant itself: a loop that lets the model call read/write tools against real (fixture) data, with a rail on every edge of the loop and an approval gate in front of anything that changes state.



TOOL	KIND	NOTES
`search_documents`	read	the RAG path — what it returns becomes the grounding context
`lookup_fee`	read	published fee table
`check_claim_status`	read	declares `reference` in `unmask_args`
`file_grievance`	write	**stops for approval**, resolves the claim reference it files against

Three locks, each answering a failure mode someone has already hit

- `agent.approval_required_for` — every write tool asks a person before it runs. The approval card shows the *unmasked* summary, because a vault token is not something a person can meaningfully consent to. Nothing runs until they answer, and the decision itself joins the trace.
- `agent.tool_result_trust` — a tool result crosses `agent.data` before the model reads it. A record field somebody else filled in is attacker-reachable text, exactly like a user message.
- `agent.vault_unmask_scope` — a tool declares which arguments it may see raw. The model can *ask* for a lookup; it does not get to decide who is entitled to see the identifier behind it.

```
POST /api/agent/chat       {message, session_id}  →  reply, or a pending approval
POST /api/agent/approve    {token, approved}      →  resumes the paused turn
GET  /api/agent/tools                             →  the tool set, with its own gates
```

6 · Where the agents work — layer two: the supervisor pipeline

A second, deeper agent path, in `guardrails/agents/`, alongside `AgentRunner`. Where the rail stack asks "what does this text match," this layer asks a model to *reason* about the request as a whole — and it is built so that reasoning is the last resort, not the first check.

6.1 · `GuardrailSupervisor` — flat, cheap, mostly deterministic

A single-hop loop over six flat tools, with two features neither `Supervisor` nor any specialist has on its own: a zero-judge hard-block pre-check, and a deterministic risk-band gate around the one judge call that remains.

```
PRECHECK ──► detect_prompt_injection / detect_destructive_intent  (deterministic, no judge)
   │
   ├─ hit ──► HARD_BLOCK ──► ENFORCE ──► ACT ──► TRACE          (judge never called)
   │
   └─ clear
       │
       ▼
     PLAN (judge)  which of 6 tools are worth running, on this request?
       │
       ▼
     SELECT + EXECUTE   the named tools run; OBSERVE collects what they returned
       │
       ▼
     DECIDE — risk_proxy = max(tool confidence), computed in plain code
       │
       ├─ below risk_low_threshold   ──► deterministic ALLOW      (no judge)
       ├─ above risk_high_threshold  ──► deterministic BLOCK/MASK (no judge)
       └─ inside the band            ──► one judge call decides
       │
       ▼
     ENFORCE — PolicyEngine.decide() combines the recommendation with a config floor
       │
       ▼
     ACT ──► TRACE
```

The six tools it may plan: `detect_pii`, `detect_prompt_injection`, `detect_destructive_intent`, `check_scope`, `check_semantic_risk`, `get_policy`. Its own docstring states the precedence it enforces, top to bottom: system security constraints (a fixed, six-value action type; eleven forbidden capability names denied unconditionally) → the hard-block pre-check and the risk-band gate → RBAC, when a caller supplies an authorization context → the guardrail policy floor → the judge's own recommendation → the user's request, which carries no independent weight of its own.

6.2 · `Supervisor` — the deeper pass, six specialist agents

```
ANALYZE + PLAN (judge)   which of 6 registered specialists does this request need?
      |
      ▼
SELECT + EXECUTE         each named specialist runs its OWN full PLAN → DECIDE →
                          POLICY loop, sequentially, unmodified — not a duplicate,
                          not called concurrently the way rail-stack jobs are
      |
      ▼
OBSERVE                  the specialists' own structured AgentResults, nothing else
      |
      ▼
EVALUATE + DECIDE         0 agents ran → ALLOW, no judge call
                          1 agent ran  → its decision stands, un-relitigated
                          2+ agents ran → a second judge call weighs them
      |
      ▼
POLICY   PolicyEngine.decide() against a floor built from what each selected
         specialist's OWN Policy Engine already enforced — the supervisor can be
         more restrictive than every agent it reconciles, never less
      |
      ▼
ACT      PIICapabilities.execute() — the same boundary a standalone specialist uses
```

SPECIALIST	WATCHES FOR
`pii`	personal data anywhere in the request, including a question that only references someone's record
`injection`	wording aimed at the assistant's own behaviour, not merely a heavy or frustrated subject
`content`	hate, violence, self-harm intent, sexual content out of place — not distress or bereavement on their own
`scope`	whether the request belongs to this assistant's actual domain at all
`authorization`	a resource that could belong to someone other than the caller, resolved against the caller's own permissions
`grounding`	claims a generated answer makes that the retrieved context does not support

They are planned selectively and run **sequentially** — unlike the rail stack's own jobs, which share one deadline and run in parallel. The common case is that most requests need none of them, and a specialist the plan never selects costs nothing at all.

6.3 · `PolicyEngine` — the one place with the final word

Stateless, deterministic, no judge call: a lookup and a comparison. Every specialist's own judge-backed recommendation is combined with a policy floor read straight from `config/policy.yaml` — the same key a deterministic rail would read for that surface — and the **more restrictive** of the two always wins.

```
ACTION_RANK   ALLOW(0) < FLAG(1) < MASK(2) < REDACT(3) < BLOCK(4)
ESCALATE is not ranked on this scale — it means "defer to a human," and wins outright
against anything but a confident floor from the other side.

final = max(recommended_action, floor_from_policy)   — never softer than the floor
```

> **An agent recommending ALLOW cannot override policy.** If `config/policy.yaml` says a checksum-verified SSN must be masked, a judge that recommends ALLOW is overruled by the floor. An agent recommending BLOCK is always free to be stricter than its floor requires — more caution never needs anyone's permission.

7 · How one request actually moves through both layers

`POST /api/pipeline/run {text, surface}` is where the whole review comes together — it chains the deep-reasoning layer in front of the deterministic rail stack, in order, and only reaches the model at all if nothing upstream already stopped the request:

```
POST /api/pipeline/run   {text, surface}

  GuardrailSupervisor        cheap, fast, mostly deterministic. A hard-block hit stops
                              here — the judge, and every layer below, never runs at all.
        |  (only if not already stopped, and a model is configured)
        ▼
  Supervisor                 the deeper pass. One judge call plans which specialists are
                              worth running; each runs; a second judge call weighs the
                              results if more than one ran.
        |
        ▼
  PolicyEngine.decide()      combines both layers' already-enforced actions into one
                              floor — the same "recommendation + floor" rule every rail
                              in this stack already uses
        |  (only if the combined action is not BLOCK / REDACT / ESCALATE)
        ▼
  Engine.converse()          the real retrieval → LLM → output-rails → grounding
                              pipeline from §2.2 and §3, unmodified
```

Measured, one real request — "what are the Objects of Tamil Nadu Cooperative Union?", nothing risky, both layers correctly found nothing extra to add:

STAGE	WALL CLOCK	WHAT IT SPENT IT ON
`GuardrailSupervisor`	`5.7s`	one PLAN judge call
`Supervisor`	`3.4s`	one plan call, selected zero specialists
`Engine.converse()`	`19.0s`	its own six sequential stages, each parallel internally
Total	`28.1s`	retrieval itself was 1.6ms of that — the two *planning* calls are almost the whole cost

> **Worth saying out loud in a demo.** A slow response here usually does not mean retrieval is slow — `corpus.search()` ran in 1.6ms in the measurement above. It means the planning calls upstream of it are, because reasoning about whether to look further is exactly the part this design tries to make as rare as possible, and still isn't free when it does run.

The response names which stage ended the request, so a short response is never a mystery: `stopped_at` is `"guardrail_supervisor"`, `"policy_engine"`, or `null` — meaning it reached `Engine.converse()` and ran the full chain from §3.

8 · The six demo scenarios

Each one drives the real engine — not a recording — and asserts on what actually came back. Runnable live with `POST /api/scenarios/{id}/run`.

	SCENARIO	WHAT IT PROVES
simple	`clean`	every rail runs and passes; the answer is grounded
simple	`pii`	masking is not refusing — the model works on tokens, the user loses nothing
simple	`injection`	the pattern layer blocks pre-model in under a millisecond, and the refusal never names the technique
complex	`poisoned-doc`	the same payload caught on two surfaces — quarantined at ingestion, withheld at `agent.data` when it arrives through a tool instead
complex	`agentic-claim`	a vaulted identifier the model never sees, a tool entitled to resolve it, a write that stops for a person, the real reference on the filed record
simple	`resident-record`	a retrieved record's PII never resolves for whoever merely asked — the deliberate contrast with `pii`, where the caller owns the value and gets it back

A minimal script for walking someone through it

1. Ask a clean question — point at the trace, note every stage passed and the answer is grounded.
2. Ask a question containing an SSN — point out the reply still works and the value comes back correctly for the same caller, while the trace shows a masked token reached the model.
3. Try a prompt-injection phrase — point out the **0.1ms** latency in the trace: the pattern layer caught it before any judge call, and the refusal never names the technique that was matched.
4. Run `agentic-claim` — show the approval card carrying the unmasked summary, decline it once, then approve it and show the filed reference.
5. Run `poisoned-doc` — show the same payload quarantined on upload, then show it withheld again when it arrives through a tool result instead.

9 · Reference — layout and commands

```
backend/guardrails/
├── engine.py                 the pipeline: evaluate(), converse(), ingest()
├── registry.py                every tunable parameter, declared once
├── rails/                     DECIDE ABOUT TEXT — read a string, return a verdict
│   ├── normalize.py · words.py · pii.py · entities.py · presidio_ner.py
│   ├── content.py · grounding.py · policy.py · adjudicator.py
├── agent/                     DECIDES WHAT TO DO NEXT — layer one, §5
│   ├── runner.py               the AgentRunner tool loop
│   └── tools.py                what it may call, and what each may see unmasked
├── agents/                    the deeper path — layer two, §6
│   ├── guardrail_supervisor.py     flat, fast, mostly-deterministic precheck
│   ├── supervisor.py               plans and runs up to six specialists
│   ├── policy_engine.py            combines every layer's verdict into one floor
│   └── pii_agent.py · injection_agent.py · content_safety_agent.py ·
│       scope_agent.py · authorization_agent.py · grounding_agent.py
└── knowledge/                 seed.py, seed_documents/, ingest.py

frontend/demo/
├── stages.html                the chat request, stage by stage, live trace overlaid
├── flow.py                    the same chart, in the terminal
└── README.md                  the mermaid version of the diagrams in §2
```

Run it locally

```
python run.py                          # start the server — http://127.0.0.1:8000
python run.py --ask "..."              # one request through the stack, printed as a trace
python frontend/demo/flow.py --sample injection
python run.py --eval                   # score against the labelled eval suite
python -m pytest tests/ -q             # the regression suite

# the two-layer pipeline from §7, over HTTP, signed in:
POST /api/pipeline/run   {"text": "...", "surface": "user.prompt"}
POST /api/scenarios/{id}/run
```

Guardrail Console — Project Demo Review · generated from the repository at `E:\GuardRails` · see also `README.md`, `docs/guardrails-explained.pdf`, and `docs/architecture.pdf` for adjacent detail this review does not repeat.